INTRODUCTION TO ARTIFICIAL INTELLIGENCE

DECISION TREE ASSIGNMENT

Ayben GÜLNAR-191180041

JANUARY 2023

CONTENT

1.INTRODUCTION 2
2. CODES 2
3. MY TREE OUTPUT 8
4. DECISION TREE VISUALIZATION 10

1.INTRODUCTION

Öncelikli olarak ödevde birkaç madde olduğu için gerçekleştirdiklerimi yazayım.

- *N özellik sayısını 615 satırlı 13 sütunlu yeni bir dataset kullanarak genişlettim.*
- *M sayısını dataset büyüklüğü dahilinde kullanıcıdan aldım.*
- *Decision treeyi herhangi bir kütüphane kullanmadan kodladım.*
- *Ağacımı görselleştirdim.*
- *Her özellik için hesaplamalar ve elde edilen sonuçlar adım adım gösterdim.*

First, I converted the values in my dataset to numerical values, and then I began my operations.

```
In [53]:  d = pd.read_csv("dataset.csv")
          d.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   loan_id            614 non-null    object
 1   gender             601 non-null    object
 2   married            611 non-null    object
 3   dependents         599 non-null    object
 4   education          614 non-null    object
 5   self_employed      582 non-null    object
 6   applicantincome    614 non-null    int64
 7   coapplicantincome  614 non-null    float64
 8   loanamount         592 non-null    float64
 9   loan_amount_term   600 non-null    float64
 10  credit_history     564 non-null    float64
 11  property_area      614 non-null    object
 12  loan_status        614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

```
In [55]:  #column name inorder:
          #loan_id,gender,married,dependents,education,self_employed,applicantincome,coapplicantincome,loanamount,loan_amount_term,crea

          data = pd.read_csv("numerikdataset.csv", skiprows=1, header=None)
          data = data.drop(0, axis=1)
          data = data.dropna()

          m = int(input("615'den küçük M değerini giriniz: "))

          data = data.head(m)
          data

615'den küçük M değerini giriniz: 300
```

Out[55]:

	1	2	3	4	5	6	7	8	9	10	11	12
1	0.0	1.0	1	1	0.0	4583	1508.0	128.0	360.0	1.0	0.0	0
2	0.0	1.0	0	1	1.0	3000	0.0	66.0	360.0	1.0	1.0	1
3	0.0	1.0	0	0	0.0	2583	2358.0	120.0	360.0	1.0	1.0	1

2. CODES

```
class Node():
    def __init__(self, feature_index=None, threshold=None, left=None,
right=None, info_gain=None, value=None):
        ''' constructor '''

        # for decision node
        self.feature_index = feature_index
        self.threshold = threshold
        self.left = left
        self.right = right
        self.info_gain = info_gain
        # for leaf node
        self.value = value
```

The Node class you have defined appears to represent a node in a decision tree. A decision tree is a machine learning model used for classification and regression tasks. The Node class has several attributes that are used to store information about the node.

The feature_index attribute represents the index of the feature being split on at this node. The threshold attribute represents the value at which the feature is being split. The left and right attributes represent the left and right children of the node, respectively. The info_gain attribute represents the information gain achieved by splitting on this node.

The value attribute is used to store the predicted class or value for this node in the case that it is a leaf node (i.e., a node with no children). In other words, it is used to store the final prediction made by the decision tree when it reaches a leaf node.

The following Code: The DecisionTreeClassifier class you have defined is a class for training a decision tree model for classification tasks. The decision tree is trained using the "ID3" (Iterative Dichotomiser 3) algorithm, which works by constructing a tree in a top-down, greedy manner. At each step, the algorithm selects the feature that maximizes the information gain, and splits the data on that feature. This process is repeated recursively on the resulting subsets until certain stopping conditions are reached, such as when the data can no longer be split or when the tree has reached its maximum depth.

The __init__ method is the constructor for the class, which initializes the root of the tree to None and sets the minimum number of samples required to split a node and the maximum depth of the tree. These are known as hyperparameters, and they control the complexity of the model.

The build_tree method is a recursive function that builds the decision tree by dividing the dataset into smaller and smaller subsets. It takes as input a dataset (dataset) and the current depth of the tree (curr_depth), and returns a Node object representing the root of the built tree.

The get_best_split method is used to find the best split of the data at each node. It takes as input the dataset and the number of samples and features in the dataset, and returns a dictionary containing the best feature index and threshold to split on, as well as the resulting left and right subsets of the data. The best split is determined by maximizing the information gain, which is a measure of how much the split improves the purity of the data.

The split method is used to divide the dataset into two subsets based on a given feature index and threshold. It takes as input the dataset and the feature index and threshold to split on, and returns the resulting left and right subsets of the data.

The information_gain method is used to compute the information gain achieved by splitting the data on a given feature. It takes as input the parent set, the left and right child sets resulting from the split, and the mode (either "entropy" or "gini"). It returns the information gain, which is computed using either the entropy or gini impurity, depending on the specified mode.

The calculate_leaf_value method is used to compute the predicted class or value for a leaf node. It takes as input the target values for the data at the leaf (Y) and returns the most common class or the mean value of the target. This is used to make a prediction when the tree reaches a leaf node.

```
class DecisionTreeClassifier():
    def __init__(self, min_samples_split=2, max_depth=2):
        ''' constructor '''

        # initialize the root of the tree
        self.root = None

        # stopping conditions
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth

    def build_tree(self, dataset, curr_depth=0):
        ''' recursive function to build the tree '''

        X, Y = dataset[:,:-1], dataset[:,-1]
        num_samples, num_features = np.shape(X)

        # split until stopping conditions are met
        if num_samples>=self.min_samples_split and curr_depth<=self.max_depth:
            # find the best split
            best_split = self.get_best_split(dataset, num_samples,
num_features)
            # check if information gain is positive
            if best_split["info_gain"]>0:
                # recur left
                left_subtree = self.build_tree(best_split["dataset_left"],
curr_depth+1)
                # recur right
                right_subtree = self.build_tree(best_split["dataset_right"],
curr_depth+1)
                # return decision node
                return Node(best_split["feature_index"],
best_split["threshold"],
                            left_subtree, right_subtree,
best_split["info_gain"])
```

```
        # compute leaf node
        leaf_value = self.calculate_leaf_value(Y)
        # return leaf node
        return Node(value=leaf_value)

    def get_best_split(self, dataset, num_samples, num_features):
        ''' function to find the best split '''

        # dictionary to store the best split
        best_split = {}
        max_info_gain = -float("inf")

        # loop over all the features
        for feature_index in range(num_features):
            feature_values = dataset[:, feature_index]
            possible_thresholds = np.unique(feature_values)
            # loop over all the feature values present in the data
            for threshold in possible_thresholds:
                # get current split
                dataset_left, dataset_right = self.split(dataset,
feature_index, threshold)
                # check if childs are not null
                if len(dataset_left)>0 and len(dataset_right)>0:
                    y, left_y, right_y = dataset[:, -1], dataset_left[:, -1],
dataset_right[:, -1]
                    # compute information gain
                    curr_info_gain = self.information_gain(y, left_y, right_y,
"gini")
                    # update the best split if needed
                    if curr_info_gain>max_info_gain:
                        best_split["feature_index"] = feature_index
                        best_split["threshold"] = threshold
                        best_split["dataset_left"] = dataset_left
                        best_split["dataset_right"] = dataset_right
                        best_split["info_gain"] = curr_info_gain
                        max_info_gain = curr_info_gain

        # return best split
        return best_split

    def split(self, dataset, feature_index, threshold):
        ''' function to split the data '''

        dataset_left = np.array([row for row in dataset if
row[feature_index]<=threshold])
        dataset_right = np.array([row for row in dataset if
row[feature_index]>threshold])
        return dataset_left, dataset_right
```

```
    def information_gain(self, parent, l_child, r_child, mode="entropy"):
        ''' function to compute information gain '''

        weight_l = len(l_child) / len(parent)
        weight_r = len(r_child) / len(parent)
        if mode=="gini":
            gain = self.gini_index(parent) -
(weight_l*self.gini_index(l_child) + weight_r*self.gini_index(r_child))
        else:
            gain = self.entropy(parent) - (weight_l*self.entropy(l_child) +
weight_r*self.entropy(r_child))
        return gain

    def entropy(self, y):
        ''' function to compute entropy '''

        class_labels = np.unique(y)
        entropy = 0
        for cls in class_labels:
            p_cls = len(y[y == cls]) / len(y)
            entropy += -p_cls * np.log2(p_cls)
        return entropy

    def gini_index(self, y):
        ''' function to compute gini index '''

        class_labels = np.unique(y)
        gini = 0
        for cls in class_labels:
            p_cls = len(y[y == cls]) / len(y)
            gini += p_cls**2
        return 1 - gini

    def calculate_leaf_value(self, Y):
        ''' function to compute leaf node '''

        Y = list(Y)
        return max(Y, key=Y.count)

    def print_tree(self, tree=None, indent=" "):
        ''' function to print the tree '''

        if not tree:
            tree = self.root

        if tree.value is not None:
            print(tree.value)

        else:
```

```
            print("X_"+str(tree.feature_index), "<=", tree.threshold, "?",
tree.info_gain)
            print("%sleft:" % (indent), end="")
            self.print_tree(tree.left, indent + indent)
            print("%sright:" % (indent), end="")
            self.print_tree(tree.right, indent + indent)

    def fit(self, X, Y):
        ''' function to train the tree '''

        dataset = np.concatenate((X, Y), axis=1)
        self.root = self.build_tree(dataset)

    def predict(self, X):
        ''' function to predict new dataset '''

        preditions = [self.make_prediction(x, self.root) for x in X]
        return preditions

    def make_prediction(self, x, tree):
        ''' function to predict a single data point '''

        if tree.value!=None: return tree.value
        feature_val = x[tree.feature_index]
        if feature_val<=tree.threshold:
            return self.make_prediction(x, tree.left)
        else:
            return self.make_prediction(x, tree.right)
```

```
X = data.iloc[:, :-1].values
Y = data.iloc[:, -1].values.reshape(-1,1)
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=.2,
random_state=41)
```

In this code snippet, data is a pandas DataFrame containing the data to be used for training and testing the decision tree. The data is split into a feature matrix (X) and a target vector (Y). The feature matrix X is obtained by selecting all the columns in data except the last one, while the target vector Y is obtained by selecting the last column of data. The values attribute is used to convert the pandas DataFrames to NumPy arrays. The target vector Y is also reshaped using the reshape method to ensure that it has the correct shape for use in the decision tree model.

Next, the train_test_split function from scikit-learn's model_selection module is used to split the data into training and testing sets. The X_train and Y_train variables will be used to train the model, while the X_test and Y_test variables will be used to evaluate the model's performance. The test_size parameter specifies the fraction of the data to use for testing (here, 20%), and the random_state parameter is used to seed the random number generator to ensure reproducible results.

```
Y_pred = classifier.predict(X_test)
from sklearn.metrics import accuracy_score
accuracy_score(Y_test, Y_pred)
```

The code provided calculates the accuracy of a classification model on a test set. The classifier object is used to predict the labels of the samples in X_test, and the accuracy_score function is then used to compare these predicted labels to the true labels stored in Y_test. The resulting accuracy score is a metric that can be used to evaluate the performance of the classification model.

To calculate the accuracy, the accuracy_score function counts the number of samples in X_test that have the same label as predicted by the classifier, and divides this count by the total number of samples in X_test. This gives the fraction of samples that were correctly classified, which is then multiplied by 100 to obtain the accuracy as a percentage.

3. MY TREE OUTPUT

Each line represents a node in the tree, and the indentation of the lines indicates the level of the node in the tree. The node at the beginning of each line is the parent node, and the nodes indented below it are its children.

For example, the first line represents the root node of the tree, which splits on the feature X_9 with a threshold of 0.0. If X_9 is less than or equal to 0.0, the tree follows the left child, otherwise it follows the right child. Similarly, the second line represents the left child of the root node, which splits on the feature X_7 with a threshold of 128.0.

Each node is labeled with either a class or a split. If the node is a leaf node (i.e., it has no children), it is labeled with a class, which is the predicted class for that leaf node. If the node is a decision node (i.e., it has children), it is labeled with a split, which is the feature and threshold used to split the data at that node. The decision tree is trained to make predictions for classification tasks. To make a prediction for a new sample, the tree follows the splits at each

node until it reaches a leaf node, and then returns the class label at that leaf node as the prediction.

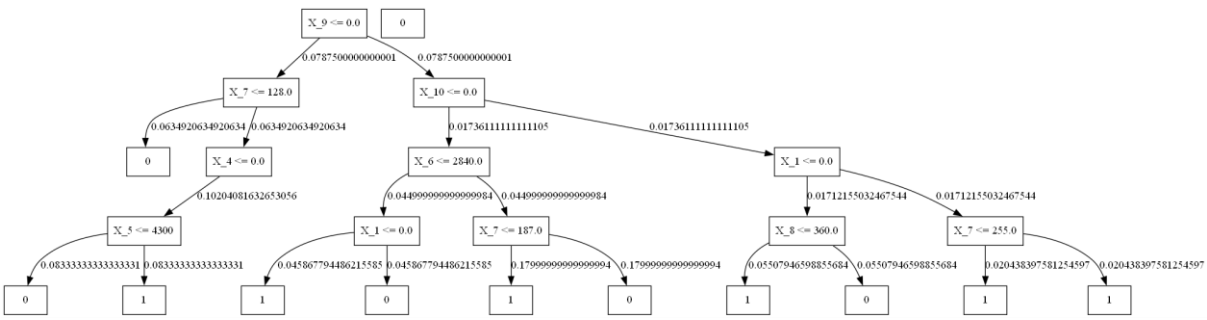
Max depth=5 output:

```
X_9 <= 0.0 ? 0.0787500000000001
 left:X_7 <= 128.0 ? 0.0634920634920634
  left:0
  right:X_4 <= 0.0 ? 0.10204081632653056
    left:X_5 <= 4300 ? 0.08333333333333331
        left:X_5 <= 2137 ? 0.375
                left:1
                right:0
        right:X_5 <= 4923 ? 0.2222222222222222
                left:1
                right:X_5 <= 14999 ? 0.4444444444444444
                                left:0
                                right:1
    right:0
 right:X_10 <= 0.0 ? 0.01736111111111105
  left:X_6 <= 2840.0 ? 0.044999999999999984
    left:X_1 <= 0.0 ? 0.045867794486215585
        left:X_5 <= 2378 ? 0.11015153984031278
                left:0
                right:X_5 <= 4000 ? 0.07750865051903122
                                left:1
                                right:1
        right:X_5 <= 5042 ? 0.06530612244897954
                left:X_6 <= 1508.0 ? 0.16253968253968257
                                left:0
                                right:1
                right:X_6 <= 1300.0 ? 0.4444444444444444
                                left:1
                                right:0
    right:X_7 <= 187.0 ? 0.17999999999999994
        left:1
        right:0
  right:X_1 <= 0.0 ? 0.01712155032467544
    left:X_8 <= 360.0 ? 0.05507946598855684
        left:X_5 <= 8566 ? 0.041205728496698624
                left:X_2 <= 0 ? 0.054032766225582896
                                left:1
                                right:0
                right:X_0 <= 0.0 ? 0.2222222222222222
                                left:0
                                right:1
        right:0
    right:X_7 <= 255.0 ? 0.020438397581254597
        left:X_5 <= 1378 ? 0.02195340501792109
                left:X_6 <= 1086.0 ? 0.4444444444444444
                                left:1
                                right:0
                right:X_5 <= 4616 ? 0.0037721123829346226
                                left:1
                                right:1
        right:X_6 <= 7101.0 ? 0.17636684303350963
                left:X_7 <= 275.0 ? 0.217687074829932
                                left:0
                                right:1
                right:0
X_9 <= 0.0 ? 0.0787500000000001
 left:X_7 <= 128.0 ? 0.0634920634920634
  left:0
  right:X_4 <= 0.0 ? 0.10204081632653056
    left:X_5 <= 4300 ? 0.08333333333333331
        left:X_5 <= 2137 ? 0.375
                  left:1
                right:0
        right:X_5 <= 4923 ? 0.2222222222222222
                left:1
                right:X_5 <= 14999 ? 0.4444444444444444
                                left:0
                                right:1
```

```
    right:0
 right:X_10 <= 0.0 ? 0.01736111111111105
  left:X_6 <= 2840.0 ? 0.044999999999999984
    left:X_1 <= 0.0 ? 0.045867794486215585
        left:X_5 <= 2378 ? 0.11015153984031278
                left:0
                right:X_5 <= 4000 ? 0.07750865051903122
                                left:1
                                right:1
        right:X_5 <= 5042 ? 0.06530612244897954
                left:X_6 <= 1508.0 ? 0.16253968253968257
                                left:0
                                right:1
                right:X_6 <= 1300.0 ? 0.4444444444444444
                                left:1
                                right:0
    right:X_7 <= 187.0 ? 0.17999999999999994
        left:1
        right:0
  right:X_1 <= 0.0 ? 0.01712155032467544
    left:X_8 <= 360.0 ? 0.05507946598855684
        left:X_5 <= 8566 ? 0.041205728496698624
                left:X_2 <= 0 ? 0.054032766225582896
                                left:1
                                right:0
                right:X_0 <= 0.0 ? 0.2222222222222222
                                left:0
                                right:1
        right:0
    right:X_7 <= 255.0 ? 0.020438397581254597
        left:X_5 <= 1378 ? 0.02195340501792109
                left:X_6 <= 1086.0 ? 0.4444444444444444
                                left:1
                                right:0
                right:X_5 <= 4616 ? 0.0037721123829346226
                                left:1
                                right:1
        right:X_6 <= 7101.0 ? 0.17636684303350963
                left:X_7 <= 275.0 ? 0.217687074829932
                                left:0
                                right:1
                right:0
```

4. DECISION TREE VISUALIZATION

Maxdepth= 3 tree:



Conclusion: 0= Loan Status: No, 1= Loan Status: Yes, X_0, X_1,X_2 = Attributes